Vulnerability Assessment su DVWA

Corso: Cybersecurity Analyst
Modulo: W13D4 (Pratica)
Piattaforma: Epicode
Studente: Emanuele Cestari
Data: 19/03/2026

—

1. Obiettivo del laboratorio

L'obiettivo di questo laboratorio è configurare un ambiente virtuale e sfruttare le vulnerabilità presenti nell'applicazione DVWA (Damn Vulnerable Web Application) utilizzando Kali Linux come macchina attaccante. **Le vulnerabilità analizzate includono:**

* XSS Reflected
* SQL Injection (non blind + UNION)

—

2. Configurazione dell'ambiente

Per questo laboratorio sono state utilizzate due macchine virtuali:

* Kali Linux (macchina attaccante)
* Metasploitable2 (macchina target)

```
kali@kali: ~
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 10
00
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
 qlen 1000
    link/ether 56:ee:d8:69:43:d9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=1.15 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.317 ms
^C
--- 192.168.50.101 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.317/0.731/1.146/0.414 ms
```

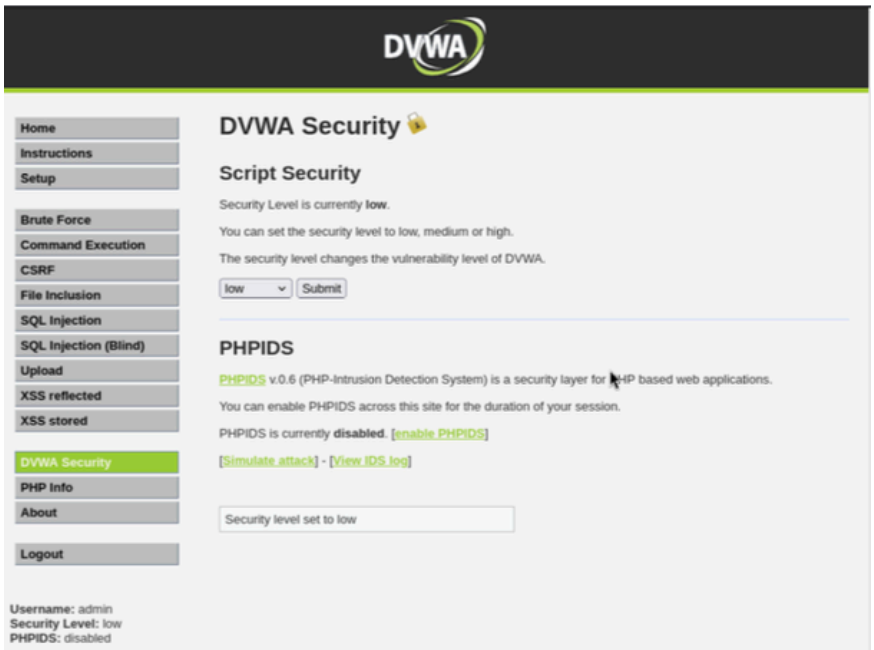
Le macchine sono state configurate nella stessa rete (ad esempio, 192.168.50.0/24). La comunicazione tra le due è stata verificata eseguendo il comando ping:

ping 192.168.50.101

Questo comando è stato eseguito sia dall'IP di Kali che dall'IP di Metasploitable, e ha avuto successo. Questo passaggio dimostra che le due macchine comunicano correttamente all'interno della rete.

Successivamente, è stato effettuato l'accesso a DVWA dal browser di Kali digitando l'URL http://192.168.50.101/dvwa. Le credenziali di accesso utilizzate sono state "admin" come username e "password" come password. L'accesso a DVWA è stato effettuato con successo, e il livello di sicurezza è stato impostato su LOW.

- Pagina DVWA
- Livello LOW

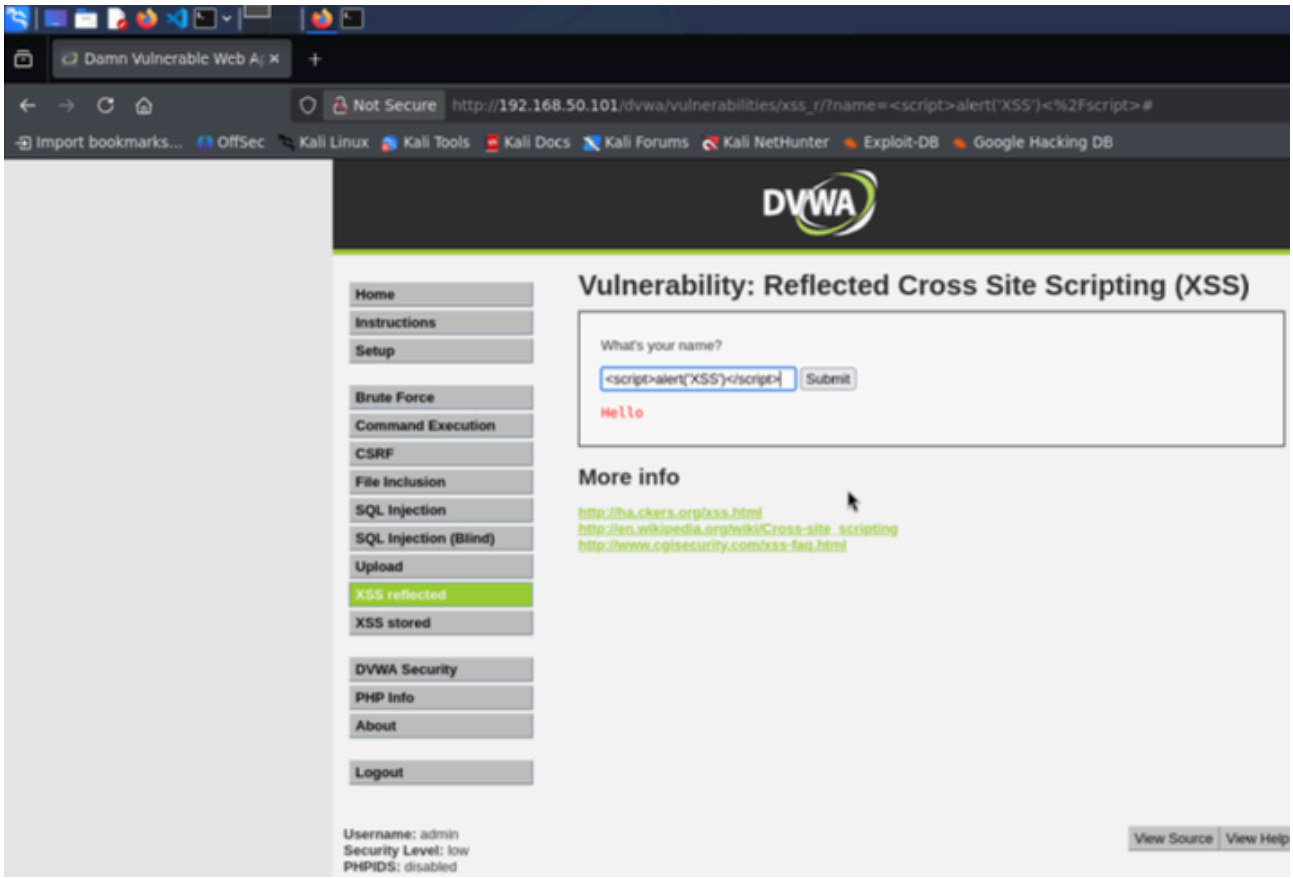


Disabilitando le protezioni con il livello LOW, si apre la strada allo sfruttamento delle vulnerabilità.

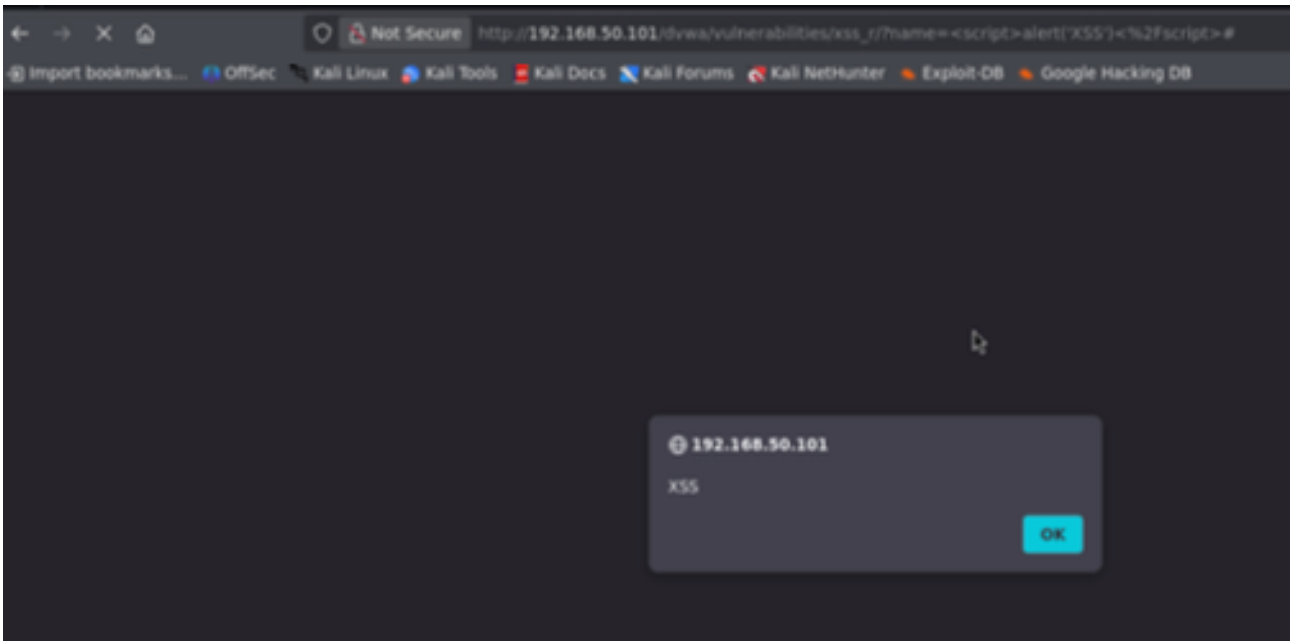
4. Vulnerabilità XSS Reflected
4.1 Test base

Payload utilizzato:
<script>alert('XSS')</script>

- Input con payload



- popup alert

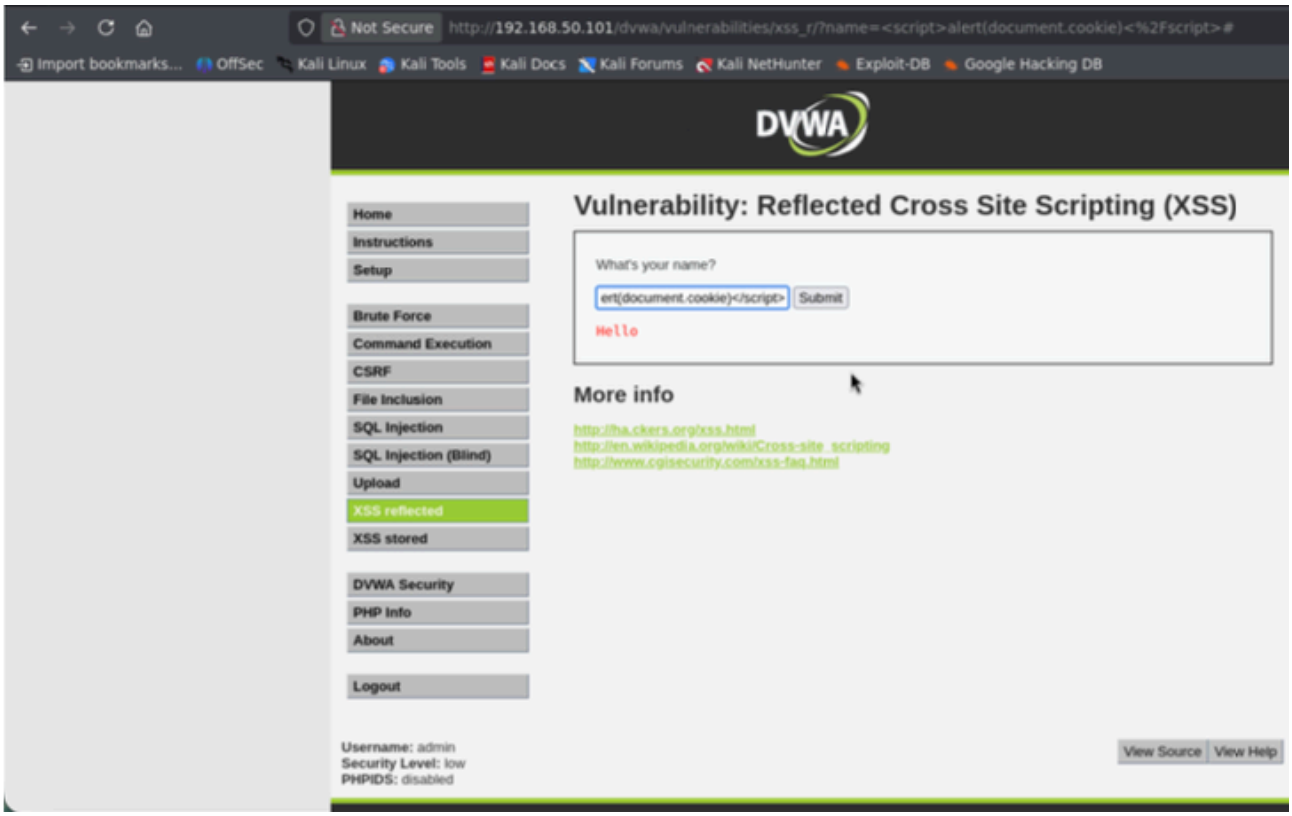


Il codice JavaScript viene eseguito nel browser, a dimostrazione che l'input non è stato filtrato.

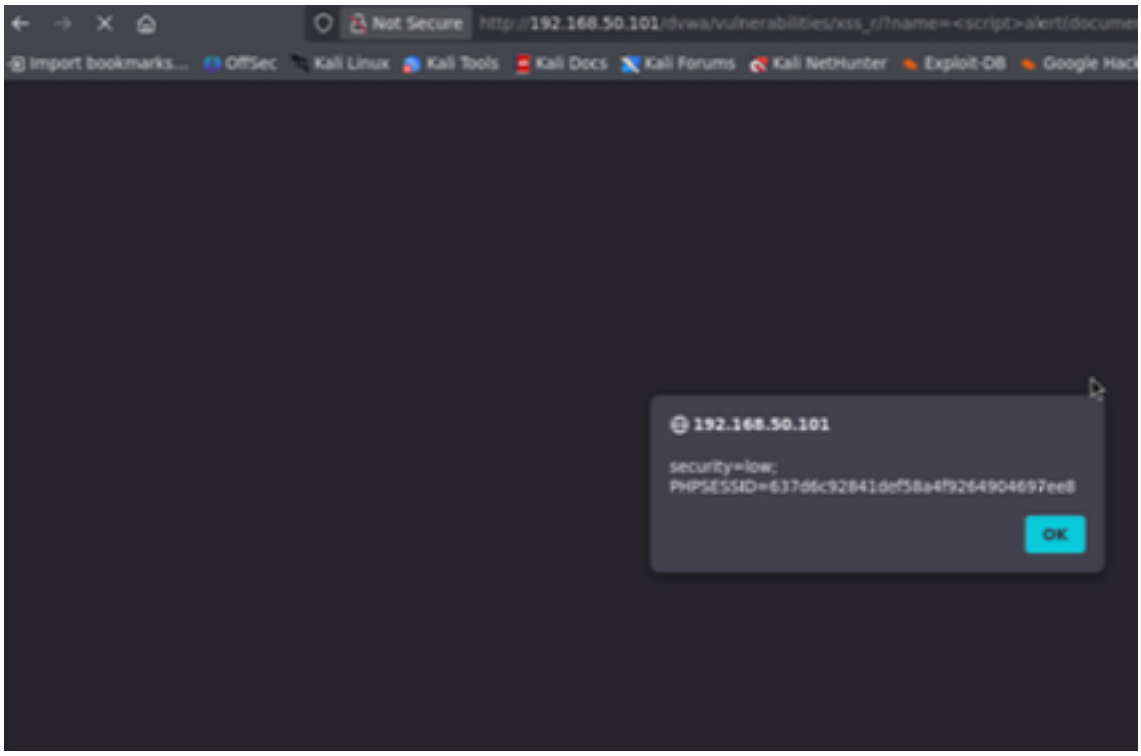
4.2 Accesso ai cookie

Payload utilizzato:
<script>alert(document.cookie)</script>

Questo payload sfrutta una vulnerabilità XSS per rubare i cookie.

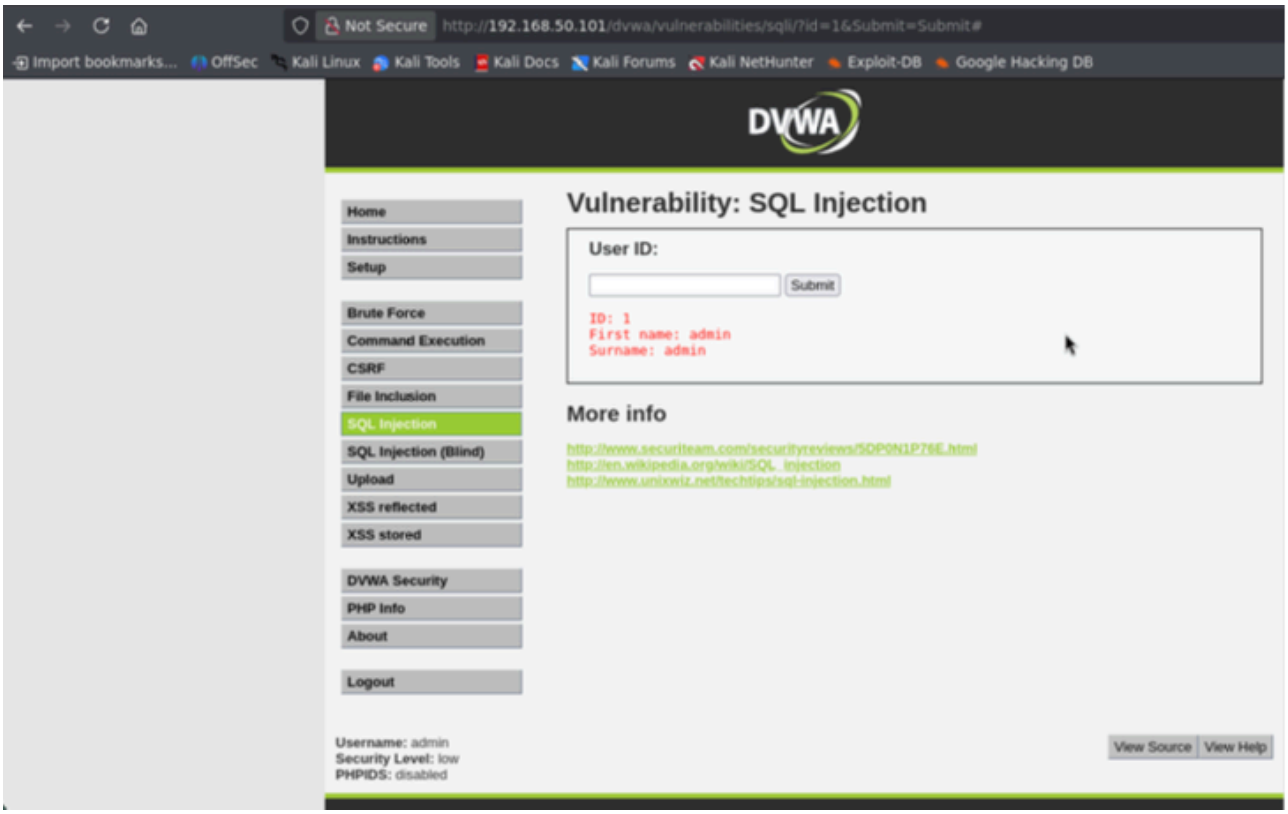


Popup con cookie



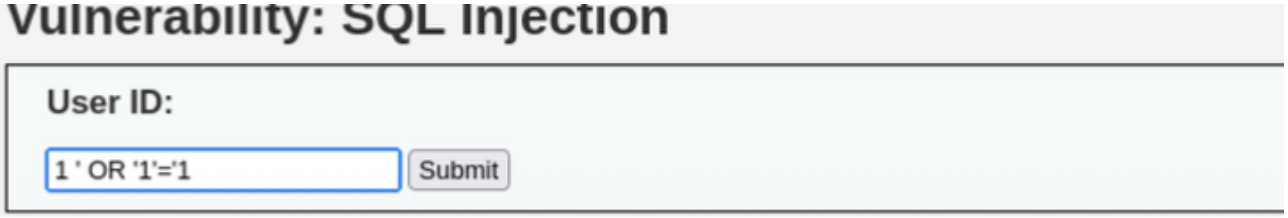
Questo fa notare che un attaccante può entrare ai cookie della sessione potenzialmente rubando le sessioni utente.

5. SQL Injection (non blind)
5.1 Test comportamento normale
Input: 1



Query normale (un solo utente)

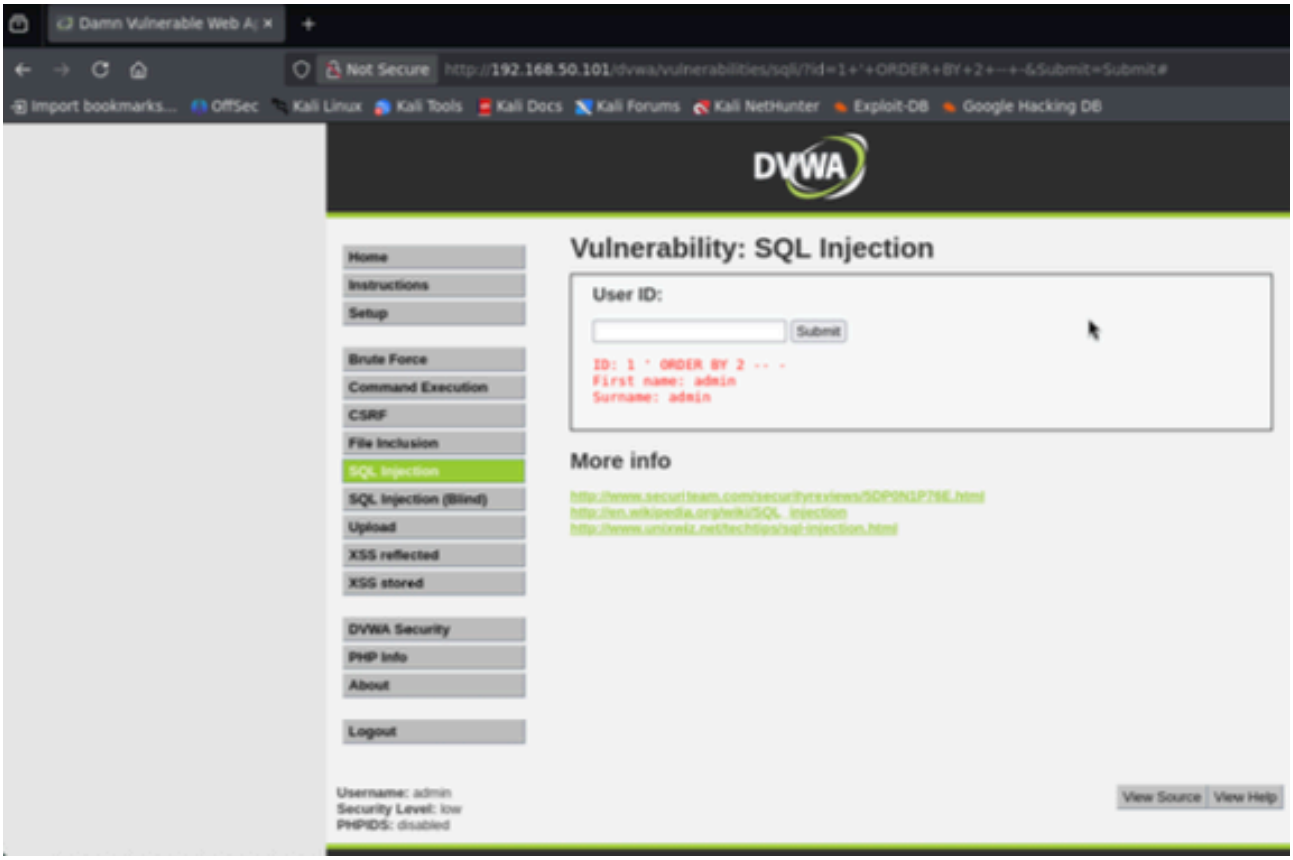
5.2 SQL Injection base
Payload:
1' OR '1'='1
SQL Injection base



5.3 Identificazione colonne

Payload:
1’ ORDER BY 2 — -

Identificazione colonne



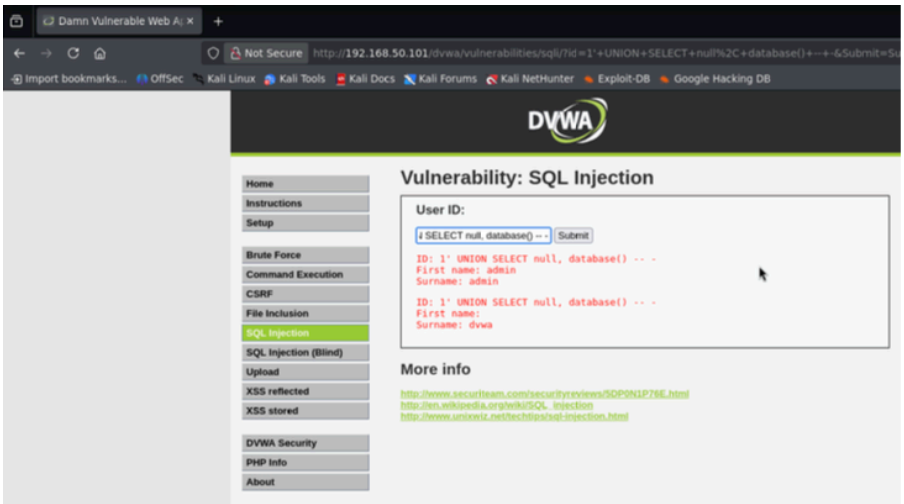
- Risultato senza errori

6. SQL Injection con UNION

6.1 Estrazione del database

Payload:
1’ UNION SELECT null, database() — -

Estrazione del nome del database:
- Il valore restituito è “dvwa”.

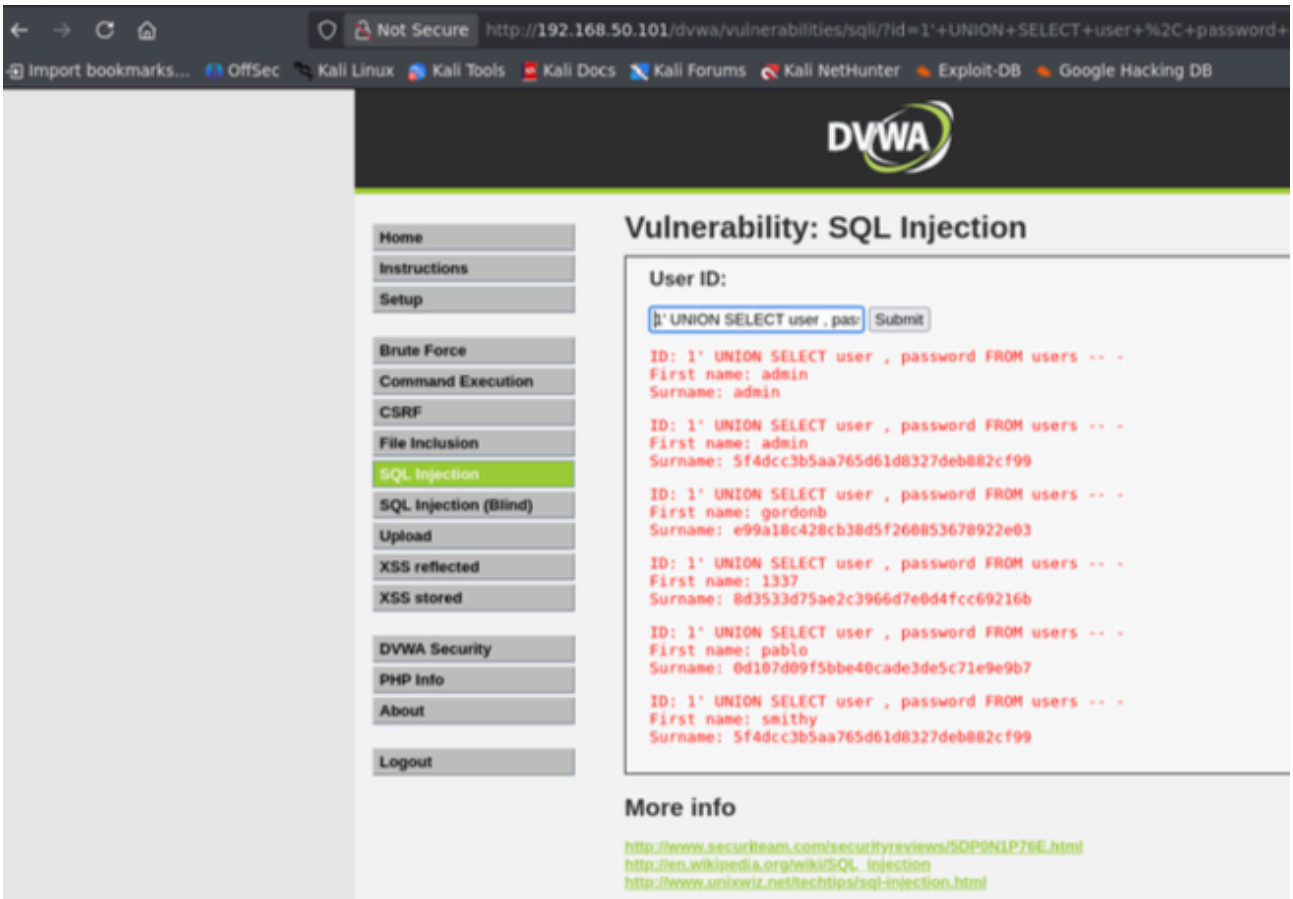


Estrazione utenti e password

Payload:
1’ UNION SELECT user, password FROM users — -

Estrazione credenziali:
- username + hash password

Il



laboratorio ha dimostrato come vulnerabilità comuni possano essere sfruttate con facilità. Sono state identificate due vulnerabilità principali: XSS (Cross-Site Scripting), che consente l’esecuzione di codice lato client, e SQL Injection, che permette l’accesso non autorizzato al database**.**

Per prevenire questi attacchi, è fondamentale implementare diverse misure di sicurezza. La validazione input aiuta a garantire che i dati forniti dagli utenti siano validi e conformi ai requisiti previsti. La sanitizzazione output rimuove o neutralizza eventuali caratteri dannosi o script inseriti nei dati in uscita. L’utilizzo di query parametrizzate e prepared statements **aiuta a prevenire le SQL Injection separando i dati dagli istruzioni SQL, rendendo più difficile per gli aggressori manipolare le query.**

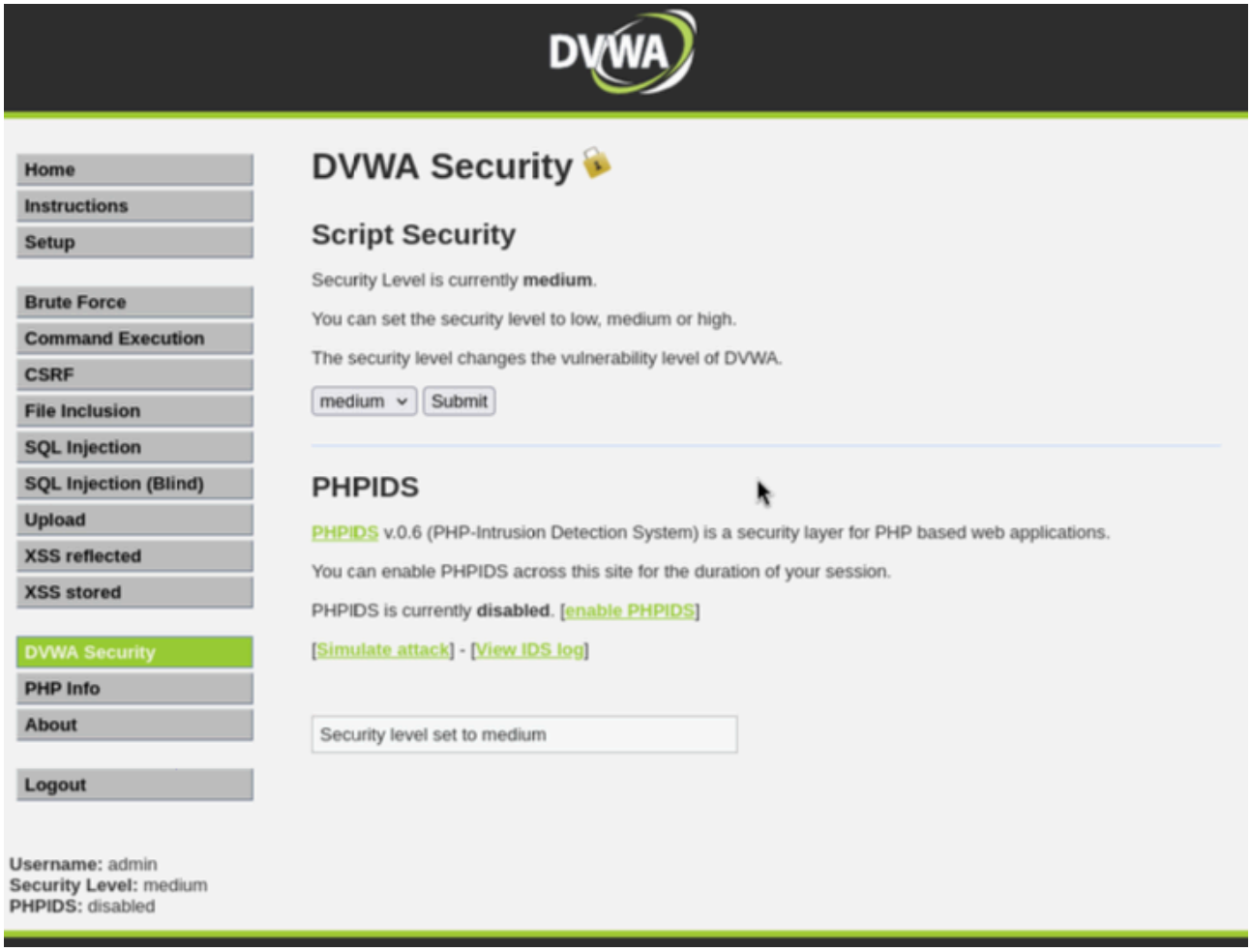
In conclusione, questa attività ha fornito una comprensione pratica dei rischi legati alla sicurezza delle web application e ha sottolineato l’importanza di implementare misure di difesa adeguate per proteggere i dati sensibili e mantenere la fiducia degli utenti.

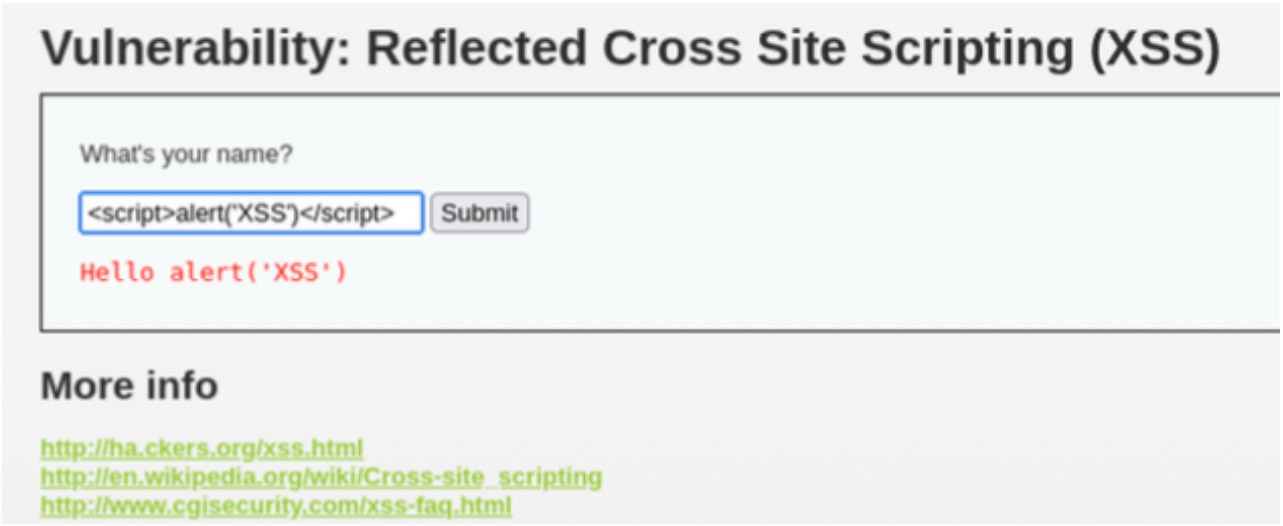
Analisi dei livelli di sicurezza (Facoltativo)

10.1 Livello MEDIUM

In questa fase, il livello di sicurezza di DVWA è stato impostato su MEDIUM per valutare l'efficacia dei controlli di sicurezza rispetto al livello LOW.

XSS Reflected (MEDIUM)





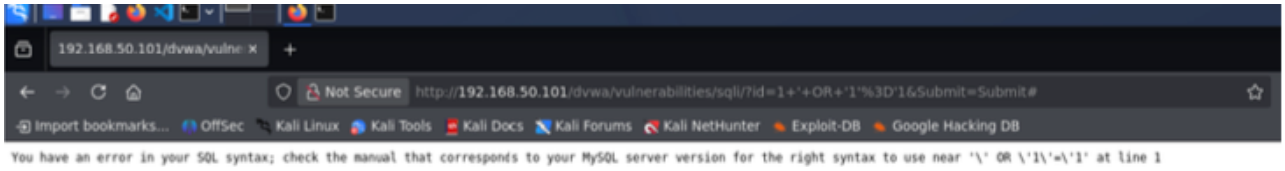
Il payload “<script>alert(‘XSS’)</script>” viene filtrato e visualizzato come testo, anziché essere eseguito. Questo suggerisce l’implementazione di meccanismi di sanitizzazione dell’input per prevenire l’esecuzione diretta di codice JavaScript. Tuttavia, il filtraggio potrebbe non essere infallibile e potrebbe essere aggirato con tecniche più sofisticate.
Bypass del filtro:

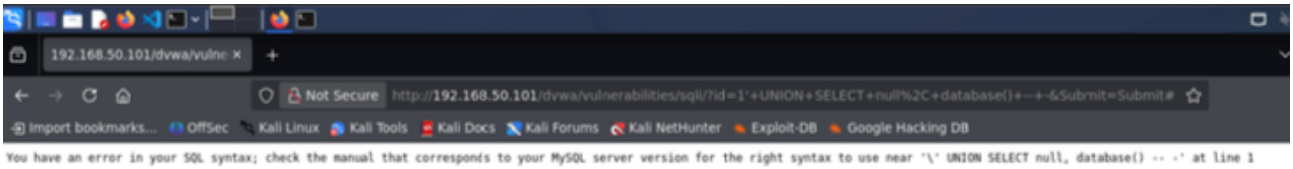
Nonostante il filtro blocchi il tag <script> in formato standard, è possibile aggirarlo modificando il case del tag HTML.

Ad esempio, utilizzando il payload:
<SCRIPT>alert(1)</SCRIPT>

Il codice viene eseguito correttamente, dimostrando che il filtro è case-sensitive e quindi facilmente bypassabile.

Questo evidenzia che la protezione a livello MEDIUM non è completa.





SQL Injection (MEDIUM)

A livello MEDIUM, l'applicazione applica un filtro che rimuove gli apici dall'input, impedendo l'utilizzo dei payload classici come ' OR '1'='1.

Tuttavia, è possibile aggirare questa protezione utilizzando tecniche alternative che non richiedono apici.

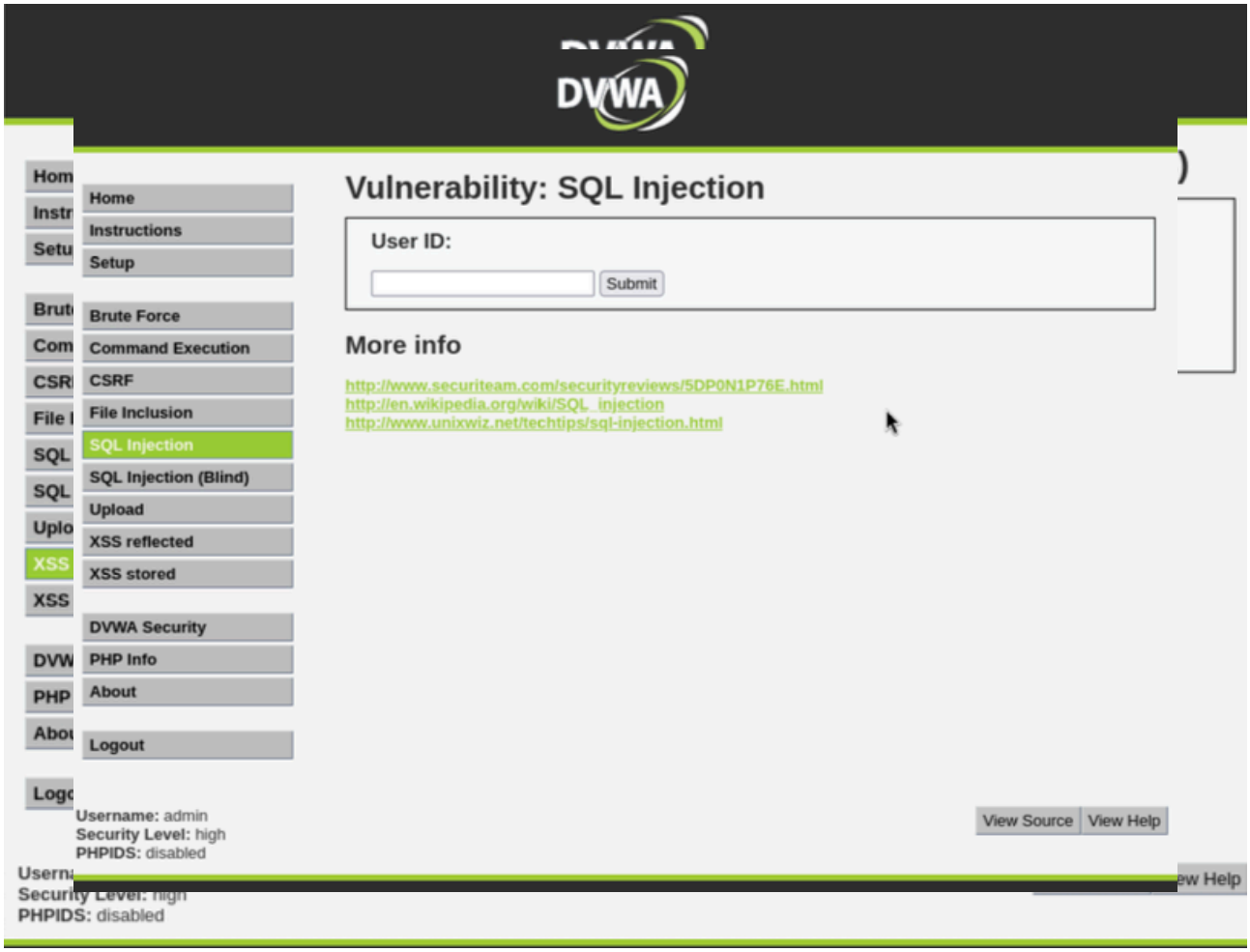
Ad esempio, il seguente payload:
5 UNION SELECT user, password FROM users #

permette comunque di estrarre dati dal database.

Questo dimostra che la SQL Injection non è completamente bloccata, ma solo parzialmente mitigata.10.2 Livello HIGH

In seguito, il livello di sicurezza è stato elevato a HIGH per garantire una protezione più avanzata.

XSS Reflected (HIGH
XSS completamente filtrato (livello HIGH)
SQL Injection (HIGH)



SQL Injection bloccata (livello HIGH)
A livello HIGH, i tentativi di SQL Injection vengono bloccati e non producono risultati utili. L'applicazione implementa controlli più rigorosi che impediscono la manipolazione delle query SQL. Questo sottolinea l'importanza di utilizzare query parametrizzate e prepared statements per prevenire attacchi di SQL Injection.

Conclusione del facoltativo:
L'analisi dei diversi livelli di sicurezza dimostra come l'implementazione di controlli progressivi migliori significativamente la protezione dell'applicazione.

- A livello LOW, le vulnerabilità sono completamente sfruttabili.
-
A livello MEDIUM, sono presenti mitigazioni parziali, ma le vulnerabilità risultano ancora aggirabili tramite tecniche di bypass dei filtri.
A livello HIGH, le vulnerabilità risultano efficacemente bloccate.

Questo evidenzia l'importanza di adottare pratiche di sviluppo sicuro per proteggere le web application da attacchi informatici. Il payload XSS non viene eseguito e viene mostrato come semplice testo, dimostrando una sanitizzazione avanzata dell'input che impedisce completamente l'esecuzione di codice JavaScript. Questo comportamento fa vedere l'efficacia di tecniche come l'escaping dell'output e la validazione rigorosa degli input